# MMA eseménykezelő és webshop

Projektdokumentáció

Angular – TypeScript – Express – MongoDB

## 1. A projekt célja

Az alkalmazás egy magyar nyelvű, küzdősportokkal foglalkozó weboldal. A látogatók bajnokokat és eseményeket böngészhetnek, illetve egy bemutató webshopban sportfelszereléseket helyezhetnek a kosárba. A fejlesztés célja egy áttekinthető, többoldalas alkalmazás létrehozása, valamint a frontend, a backend és az adatbázis összekapcsolásának bemutatása.

A projekt gyakornoki portfólióként is használható: a felületépítés mellett HTTP-kommunikációt, szűrést, állapotkezelést, adatmodellezést és készletellenőrzést is tartalmaz. A webshop demonstrációs célú, valódi fizetés és kiszállítás nélkül. Az események között kitalált bemutatóadatok szerepelnek; az oldal nem hivatalos versenynaptár. A termékképek részben illusztrációk.

## 2. Technológiák és felépítés

| Technológia | Szerep |
|---|---|
| Angular, TypeScript | Komponensek, útvonalak és felhasználói műveletek kezelése. |
| HTML, CSS | Oldalszerkezet, reszponzív elrendezés és megjelenés. |
| Node.js, Express | API-végpontok és szerveroldali üzleti logika. |
| MongoDB, Mongoose | Adattárolás, sémák és adatbázis-műveletek. |
| dotenv | Környezeti változók betöltése a backendhez. |

A frontend és a backend külön projektmappában található. Az Angular szolgáltatások HTTP-kérésekkel érik el az API-t. A backend ellenőrzi a kapott adatokat, elvégzi az adatbázis-műveleteket, majd JSON-választ küld. A böngésző nem kapcsolódik közvetlenül a MongoDB-hez.

**Adatáramlás:** felhasználói művelet → Angular → Express API → MongoDB → válasz.

A backend fő mappái: `config` az adatbázis-kapcsolathoz, `models` az adatmodellekhez, `routes` a végpontokhoz, `services` az üzleti logikához, `data` a bemutatóadatokhoz, valamint `scripts` az adatfeltöltő programhoz. A felosztás segít abban, hogy egy feladat módosítása ne igényelje a teljes alkalmazás áttekintését.

## 3. Oldalak és navigáció

A közös fejlécből elérhető a főoldal, a bajnokok, az események és a webshop. Az oldalak közötti váltást az Angular Router kezeli. A főoldal bemutatja a tartalmat, a bajnokok oldala pedig sportolói információkat jelenít meg. A megosztott komponensek egységes megjelenést adnak. Kisebb képernyőn összecsukható menü és átrendeződő kártyák segítik a használatot.

## 4. Események és webshop

**Események.** A lista az API-ból kapja az adatokat. Minden eseményhez név, dátum, helyszín és szervezet tartozik. A kereső az esemény nevében és helyszínében keres; további lehetőség a szervezet szerinti szűrés, a dátumtartomány megadása és az időrendi rendezés. A feltételek alaphelyzetbe állíthatók. A szervezetválasztó az adatokból épül fel, így új szervezet felvételekor automatikusan bővül.

Az egyes események külön részletező oldalon is megnyithatók. A felület kezeli a betöltést, a sikertelen lekérést és az üres találati listát. Nem létező eseménynél érthető hibaüzenetet kell adnia.

**Webshop.** A termékkártyák képet, nevet, leírást, árat és készletinformációt mutatnak. A kínálatban például edzőpóló, bandázs, sporttáska, boxkesztyű és védőfelszerelés található. Az elérhető termékek kosárba helyezhetők, a mennyiségek módosíthatók.

A kosár a böngésző helyi tárhelyén is megmarad, ezért egy frissítés nem feltétlenül törli a kiválasztott tételeket. Az ott tárolt adatok azonban nem tekinthetők hiteles ár- vagy készletinformációnak: a rendelés feldolgozásakor a backend ellenőrzi az aktuális értékeket.

## 5. Adatbázis és modellek

Az alkalmazás a `mma_shop` adatbázist használja. A fő gyűjtemények a `products`, `events` és `orders`. A Mongoose-modellek meghatározzák a mezőket és az ellenőrzéseket.

| Modell | Fontosabb adatok és szabályok |
|---|---|
| Termék | Egyedi numerikus azonosító és cikkszám; név, kategória, leírás, ár, készlet és képútvonal. Az ár és a készlet nem lehet negatív. |
| Esemény | Egyedi azonosító, név, dátum, helyszín és szervezet. A dátum érvényes naptári nap, `ÉÉÉÉ-HH-NN` formátumban. |
| Rendelés | Egyedi rendelés- és kérésazonosító, tételek, mennyiségek, vásárláskori árak, végösszeg és állapot. |

A rendelés a termék nevét, cikkszámát és vásárláskori árát is eltárolja. Egy későbbi termékmódosítás ezért nem változtatja meg a korábbi rendelés tartalmát. A termékek és események kezdeti adatai a `data` mappából, külön seed programmal tölthetők az adatbázisba.

## 6. A rendelés feldolgozása

1. A frontend elküldi a termékazonosítókat, mennyiségeket és az egyedi kérésazonosítót.
2. A backend ellenőrzi a bemenetet, és felismeri a már feldolgozott kérés ismétlését.
3. Az adatbázisból kiolvassa az árakat, majd ellenőrzi és csökkenti a rendelkezésre álló készletet.
4. Kiszámolja a végösszeget, elmenti a rendelést, és visszaküldi az eredményt a friss termékadatokkal.

A készletcsökkentés és a rendelés mentése tranzakcióban történik. Ha valamelyik tétel nem teljesíthető, a módosítások visszavonhatók. Az ismételt kérésazonosító megakadályozza ugyanazon vásárlás újbóli rögzítését. A helyi MongoDB a tranzakciókhoz egytagú replikakészletként fut.

## 7. API-végpontok

| Metódus | Útvonal | Feladat |
|---|---|---|
| GET | `/api/health` | Működés ellenőrzése. |
| GET | `/api/products` | Terméklista. |
| POST | `/api/products/demo-purchase` | Rendelés rögzítése. |
| GET | `/api/events` | Eseménylista. |
| GET | `/api/events/:id` | Esemény részletei. |

A backend hibás bemenetnél, nem található erőforrásnál és elégtelen készletnél megfelelő HTTP-státusszal jelez hibát. A frontendnek ezekből a felhasználó számára érthető üzenetet kell megjelenítenie.

## 8. Helyi indítás

Előfeltétel a Node.js, az npm és egy futó MongoDB-példány. A helyi adatbázis a `27018` porton, `mma-rs` nevű replikakészletként fut. A backend `.env` fájljának példája:

```
MONGODB_URI=mongodb://127.0.0.1:27018/mma_shop?replicaSet=mma-rs&directConnection=true
PORT=3001
```

A backend mappájában a függőségek telepítése, az adatfeltöltés és a fejlesztői szerver indítása:

```
npm install
npm run seed
npm run dev
```

A kialakított seed a hiányzó azonosítójú termékeket és eseményeket beszúrja. A már meglévő rekordokat és készleteket nem írja felül; egy korábbi rekord módosítását külön kell elvégezni.

Az Angular projekt mappájában, külön terminálban:

```
npm install
npm start
```

A frontend fejlesztői proxyja az `/api` kezdetű kéréseket továbbítja a backendhez. A proxy célportjának egyeznie kell a backend beállított portjával. A `/api/health` végponttal ellenőrizhető a szerver elérhetősége.

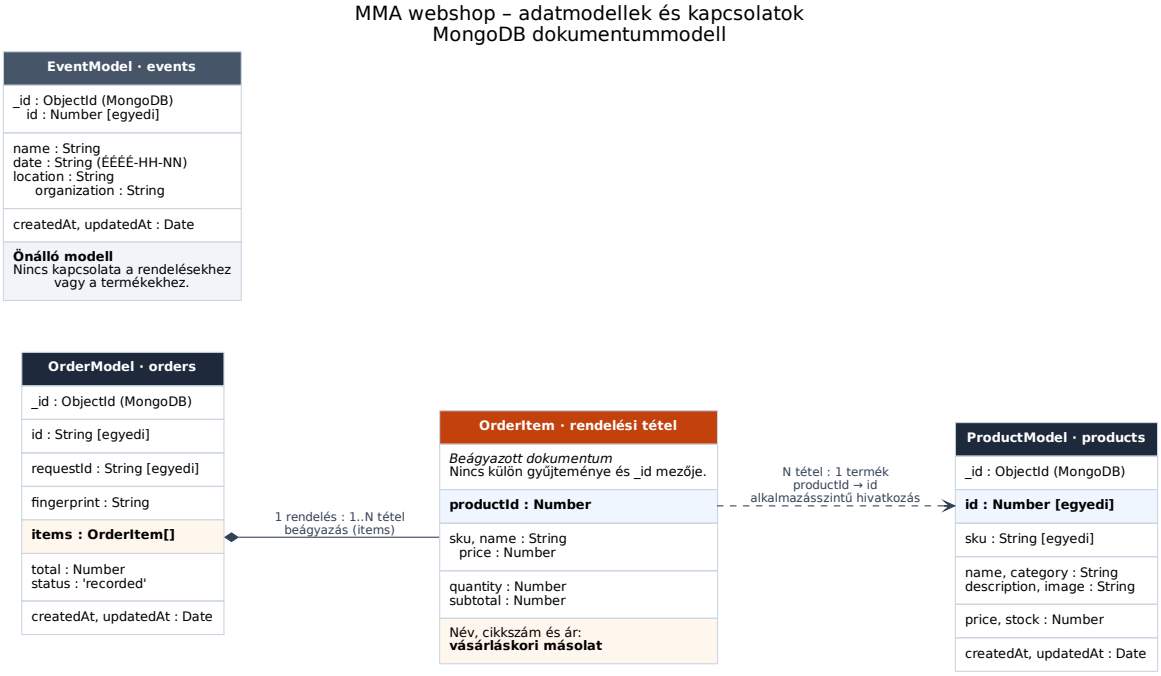
## 9. Ellenőrzés és továbbfejlesztés

A kézi ellenőrzés fontos esetei: mobilos navigáció, kombinált eseményszűrés, üres találati lista, nem létező esemény, kosár megmaradása frissítés után, elfogyott termék és rendelés utáni készletváltozás. Vizsgálandó az ismételten elküldött rendelés, valamint több egyidejű vásárlás kezelése is. Ez ellenőrzési javaslat, nem egy teljes körű tesztelés jegyzőkönyve.

A használhatóságot feliratozott mezők, látható billentyűzetfókusz, betöltési jelzések és érthető hibaüzenetek támogatják. Következő fejlesztésként adminisztrációs felület, bejelentkezés, rendelési előzmények és lapozás készíthető. Az adminisztráció bevezetéséhez szerveroldali jogosultság-ellenőrzés is szükséges.

# 10. Adatmodellek kapcsolatai

Az alábbi ábra a dokumentációban leírt Mongoose-modelleket mutatja. A rendelési tétel beágyazott dokumentum, ezért nem külön MongoDB-gyűjtemény.




**Rendelés és tétel – 1:N, beágyazás.** Egy rendelés legalább egy tételt tartalmaz az `items` tömbben. Minden ilyen tétel az adott rendelés része. A tételséma `_id: false` beállítású, ezért nincs saját MongoDB-azonosítója. Az ábrán a teli rombusz a tartalmazást jelöli.

**Tétel és termék – N:1, logikai hivatkozás.** A tétel `productId` mezője a termék egyedi, numerikus `id` mezőjére utal, nem a MongoDB `_id` mezőjére. Egy termék nulla vagy több rendelési tételben szerepelhet; egy tétel egy terméket azonosít. A szaggatott nyíl alkalmazásszintű kapcsolatot jelöl: nincs SQL-idegenkulcs vagy Mongoose `ref`. A termék meglétét és készletét a vásárlási szolgáltatás ellenőrzi.

**Rendelés és termék – közvetett N:M kapcsolat.** Egy rendelés több terméket tartalmazhat, és ugyanaz a termék több rendelésben is megjelenhet. Ezt az `items` tömb valósítja meg. A tétel a termék nevét, cikkszámát és árát vásárláskori másolatként tárolja, így a termék későbbi módosítása nem írja át a rendelést. A termék törlését a logikai hivatkozás önmagában nem tiltja.

**Esemény – önálló modell.** Az `events` gyűjteményhez jelenleg nem tartozik termék- vagy rendeléskapcsolat. Az `organization` szöveges mező; nem külön szervezetmodellre hivatkozik. A készlet a termék `stock` mezője, tehát külön raktármodell sincs. Felhasználómodell és felhasználóra hivatkozó rendelési mező sem szerepel ebben az adatmodellben.